



자연어 처리와 컴퓨터비전 심층학습 5,6장: 토큰화, 임베딩

⚙ Status	Not started
👤 Assignee	 시현 이
📅 Due date	05/12/2026
📌 Task type	개념및코드필사
≡ Description	자연어 처리와 컴퓨터비전 심층학습 5,6장: 토큰화, 임베딩

5장: 임베딩

자연어 처리(NLP): 컴퓨터가 인간의 언어를 이해하고 해석 및 생성하기 위한 기술

NLP 모델 개발 위해 필요한 문제 해결

- 모호성(Ambiguity): 맥락적인 의미 이해하고 구분
- 가변성(Variability): 사투리, 강세, 신조어, 작문 스타일로 인해 가변적인 특징을 처리 + 사용중인 언어 이해
- 구조(Structure): 구문, 의미를 해석할 수 있어야함

⇒ 이를 위해 Corpus를 Token으로 나눠야함(**Tokenization**)

→ Corpus(말뭉치): 목적에 따라 구축되는 텍스트 데이터

→ Token: 개별단어나 문장 부호와 같은 텍스트를 의미

Tokenizer: 텍스트 문자열을 토큰으로 나누는 알고리즘 또는 SW

- 입력: '형태소 분석기를 이용해 간단하게 토큰화할 수 있다.'
- 결과: ['형태소', '분석기', '를', '이용', '하', '어', '간단', '하', '게', '토큰', '화', '하', '려', '수', '있', '다', '.']

- Tokenizer 구축 방법
 - 공백 분할: 텍스트를 공백 단위로 분리해 개별 단어로 토큰화
 - 정규표현식 적용: 정규 표현식으로 특정 패턴을 식별해 텍스트로 분할
 - 어휘 사전 적용: 사전에 정의된 단어 집합을 토큰으로 사용
 - 어휘사전(Vocab) : 사전에 정의된 단어를 활용해 토큰나이저 구축
 - 없는 단어나 토큰 존재 가능(OOV(Out of Vocab))
 - 큰 어휘 사전 구축할 시
 - 학습 비용의 증대
 - 차원의 저주(토큰 개수만큼 차원이 필요)
 - 머신러닝 활용: 데이터셋을 기반으로 토큰화하는 방법을 학습한 머신러닝을 적용

단어 및 글자 토큰화

단어 토큰화

: 텍스트 데이터를 단어 단위로 분리

: 띄어쓰기, 문장 부호, 대소문자 등의 특정 구분자를 활용

- 문자열 데이터 형태는 split 메서드를 이용해 쉽게 토큰화할 수 있음
 - split 메서드는 주어진 구분자를 통해 문자열을 리스트 데이터로 나눠줌
 - 구분자 입력하지 않으면 공백을 기준

글자 토큰화

: 글자 단위로 문장을 나누는 방식

: 비교적 작은 단어 사전을 구축할 수 있음

- 언어 모델링과 같은 시퀀스 예측 작업에서 활용
- 자소 단위로 나눠서 자소 단위 토큰화 수행
 - 여기서는 자모(jamo) 라이브러리 활용

형태소 토큰화

: 텍스트를 형태소 단위로 나누는 토큰화 방법으로 언어의 문법과 구조를 고려해 단어를 분리하고 이를 의미 있는 단위로 분류하는 작업

- 한국어와 같은 교착어(Agglutinative Language)인 언어에서 중요하게 수행됨
- 품사 태깅(POS Tagging): 텍스트 데이터를 형태소 분석해 각 형태소에 해당하는 품사(POS)를 태깅하는 작업

형태소 어휘 사전

: 자연어 처리에서 사용되는 단어의 집합인 어휘 사전 중에서도 각 단어의 형태소 정보를 포함하는 사전

하위 단어 토큰화

: 형태소 분석기는 전문용어나 고유어가 많은 데이터를 처리할 때 약점을 보임

: 신조어의 발생, 오타자, 축약어 등을 분석할 때의 어려움을 해결하기 위한 방법 '하위 단어 토큰화'

- 하나의 단어가 빈번하게 사용되는 하위단어(Subword)의 조합으로 나누어 토큰화
 - ex) 'Reinforcement' → Rein/force/ment 등으로 나눠 처리

바이트 페어 인코딩(Byte Paire Encoding, BPE)

=Digram Coding

- 연속된 글자 쌍이 더 이상 나타나지 않거나 정해진 어휘 사전 크기에 도달할 때까지 조합 탐지와 부호화를 반복하며 자주 등장하는 단어를 하나의 토큰으로, 적게 등장하는 단어는 여러 토큰의 조합으로 표현

- 원문: abracadabra
 - Step #1: AracadAra
 - Step #2: ABcadAB
 - Step #3: CcadC
- 입력 데이터에서 가장 많이 등장한 글자의 빈도수 측정 → 가장 빈도수 높은 글자쌍 탐색
 - 'ab' 글자쌍 빈도수 높음 → 새로운 글자 'A'로 치환(아무 글자로 치환)
 - 'ra' 글자쌍 → 'B'로 치환
 - 'AB'를 'C'로 치환
- Corpus에서 적용 → 단어가 등장한 빈도를 계산

- 빈도 사전: ('low', 5), ('lower', 2), ('newest', 6), ('widest', 3)
- 어휘 사전: ['low', 'lower', 'newest', 'widest']

빈도 사전을 바이트 페어 인코딩으로 재구성하면

- 빈도 사전: ('l', 'o', 'w', 5), ('l', 'o', 'w', 'e', 'r', 2), ('n', 'e', 'w', 'e', 's', 't', 6), ('w', 'i', 'd', 'e', 's', 't', 3)
- 어휘 사전: ['d', 'e', 'i', 'l', 'n', 'o', 'r', 's', 't', 'w']

빈도 사전을 기준으로 가장 자주 등장한 글자 쌍을 찾음

'e','s'쌍이 'newest'에서 6번, 'widest'에서 3번 등장에 총 9번으로 가장 많이 등장 → 'es'로 병합 후 어휘 사전에 추가(동일한 과정 반복)

- 빈도 사전: ('l', 'o', 'w', 5), ('l', 'o', 'w', 'e', 'r', 2), ('n', 'e', 'w', 'es', 't', 6), ('w', 'i', 'd', 'es', 't', 3)
- 어휘 사전: ['d', 'e', 'i', 'l', 'n', 'o', 'r', 's', 't', 'w', 'es']
- 빈도 사전: ('l', 'o', 'w', 5), ('l', 'o', 'w', 'e', 'r', 2), ('n', 'e', 'w', 'est', 6), ('w', 'i', 'd', 'est', 3)
- 어휘 사전: ['d', 'e', 'i', 'l', 'n', 'o', 'r', 's', 't', 'w', 'es', 'est']

- 빈도 사전: ('low', 5), ('low', 'e', 'r', 2), ('n', 'e', 'w', 'est', 6), ('w', 'i', 'd', 'est', 3)
- 어휘 사전: ['d', 'e', 'i', 'l', 'n', 'o', 'r', 's', 't', 'w', 'es', 'est', 'lo', 'low', 'ne', 'new', 'newest', 'wi', 'wid', 'widest']

워드피스

: 빈도 기반이 아닌 확률 기반으로 글자 쌍을 병합

- 학습 과정에서 확률적인 정보를 사용
 - 모델이 새로운 하위 단어를 생성할 때 이전 하위 단어와 함께 나타날 확률을 계산해 가장 높은 확률을 가진 하위 단어로 선택

수식 5.1 글자 쌍 병합 점수 수식

$$score = \frac{f(x,y)}{f(x)f(y)}$$

- f: 빈도를 나타내는 함수 / x와 y는 병합하려는 하위 단어
 - f(x,y)는 x와 y가 조합된 글자 쌍의 빈도를 의미
- 워드피스의 어휘 사전 구축 방법
 - 가장 빈번하게 등장한 쌍 = 'e'(17번)와 's' (9번) $\Rightarrow 9/17*9 = 0.06$
 - i와 d $\Rightarrow 3/3*3 = 0.33$
 - e와 s 쌍 대신에 i와 d 쌍을 병합

- 빈도 사전: ('i', 'o', 'w', 5), ('i', 'o', 'w', 'e', 'r', 2), ('n', 'e', 'w', 'e', 's', 't', 6), ('w', 'i', 'd', 'e', 's', 't', 3)
- 어휘 사전: ['d', 'e', 'i', 'l', 'n', 'o', 'r', 's', 't', 'w']

동일한 과정 반복

6장. 임베딩

:토큰화만으로는 모델을 학습할 수 없음 → 컴퓨터는 텍스트 자체를 이해할 수 없으므로 텍스트를 숫자로 변환하는 '텍스트 벡터화'과정이 필요

- 원-핫 인코딩, 빈도 벡터화 등
 - 빈도 벡터화: 문서에서 단어의 빈도수를 세어 해당 단어의 빈도를 벡터로 표현하는 방식
 - 이 방법들은 변환 쉽고 간단한 장점 있지만, 벡터의 희소성이 큼
 - Corpus 내에 존재하는 토큰의 개수만큼의 벡터 차원을 가져야하지만, 입력 문장 내에 존재하는 토큰의 수는 그에 비해 현저히 적기 때문에 컴퓨팅 비용 증가와 차원의 저주와 같은 문제를 겪을 수 있음
 - 텍스트의 벡터가 입력 텍스트의 의미를 내포하고 있지 않기 때문에 두 문장이 의미적으로 유사하다고 해도 벡터가 유사하게 나타나지 않을 수 있음
 - 'i scream for ice cream'과 'ice cream for i scream'에 원-핫 인코딩이나 빈도 벡터화를 적용하면 둘다 [1,1,1,1,1] 벡터로 변환
 - 이 문제를 해결하기 위해 Word2Vec나 fastText 등과 같이 단어의 의미를 학습해 표현하는 워드 임베딩 기법 사용
- 워드 임베딩 기법: 단어를 고정된 길이의 실수 벡터로 표현하는 방법. 단어의 의미를 벡터 공간에서 다른 단어와의 상대적 위치로 표현해 단어 간의 관계를 추론
- 동적 임베딩 기법으로 다의어나 문맥정보 다룸

언어 모델

: 입력된 문장으로 각 문장을 생성할 수 있는 확률을 계산하는 모델

자기회귀 언어 모델

: 입력된 문장들의 조건부 확률을 이용해 다음에 올 단어를 예측

- 언어 모델에서 조건부 확률은 이전 단어들의 시퀀스가 주어졌을 때, 다음 단어의 확률을 계산

수식 6.1 언어 모델의 조건부 확률

$$P(w_i | w_1, w_2, \dots, w_{i-1}) = \frac{P(w_1, w_2, \dots, w_i)}{P(w_1, w_2, \dots, w_{i-1})}$$

- 조건부 확률의 연쇄법칙 적용하면

수식 6.2 조건부 확률의 연쇄법칙을 적용한 언어 모델의 조건부 확률

$$P(w_i | w_1, w_2, \dots, w_{i-1}) = P(w_1)P(w_2 | w_1) \dots P(w_i | w_1, w_2, \dots, w_{i-1})$$

⇒ w1을 바탕으로 다음 토큰 w2가 등장할 확률 $P(w_2 | w_1)$ 예측 → w2가 다시 입력값이 되어, 모델은 확률 $P(w_3 | w_1, w_2)$ 를 예측

통계적 언어 모델

: 언어의 통계적 구조를 이용해 문장이나 단어의 시퀀스를 생성하거나 분석

시퀀스에 대한 확률 분포를 추정해 문장의 문맥을 파악해 다음에 등장할 단어의 확률을 예측함

- 마르코프 체인을 이용해 구현됨

수식 6.3 빈도 기반의 조건부 확률

$$P(\text{만나서} | \text{안녕하세요}) = \frac{P(\text{안녕하세요 만나서})}{P(\text{안녕하세요})} = \frac{700}{1000}$$

$$P(\text{반갑습니다} | \text{안녕하세요}) = \frac{P(\text{안녕하세요 반갑습니다})}{P(\text{안녕하세요})} = \frac{100}{1000}$$

⇒ 단어의 순서와 빈도에만 기초해 문장의 확률 예측 → 불완전

⇒ 관측한 적 없는 데이터를 예측하지 못하는 '데이터 희소성(Data sparsity) 문제' 발생

GPT

- Raw Sentence: GPT는 OpenAI에서 개발한 인과적 언어 모델입니다.
- Input: GPT는 OpenAI에서 개발한
- Prediction-1: 인과적
- Prediction-2: 언어 모델입니다.

BERT

- Raw Sentence: BERT는 Google에서 개발한 마스킹된 언어 모델입니다.
- Input: Bert는 [MASK]에서 개발한 [MASK] 언어 모델입니다.
- Prediction: Google, 마스킹된

N-gram

: 입력 텍스트를 하나의 토큰 단위로 분석하지 않고 N개의 토큰을 묶어서 분석

연속된 N개의 단어를 하나의 단위로 취급해 추론, N이 1일때는 Unigram, N ==2 일 때는 Bigram, N==3 일때는 Trigram으로 부름

⇒N≥4 이면 N-gram

Unigram : <u>안녕하세요</u> 만나서 진심으로 반가워요	안녕하세요. 만나서. 진심으로. 반가워요
Bigram : <u>안녕하세요 만나서</u> 진심으로 반가워요	안녕하세요 만나서. 만나서 진심으로. 진심으로 반가워요
Trigram : <u>안녕하세요 만나서 진심으로</u> 반가워요	안녕하세요 만나서 진심으로. 만나서 진심으로 반가워요

그림 6.3 N이 1, 2, 3인 N-gram

- N-gram 언어 모델은 모든 토큰을 사용하지 않고 N-1개의 토큰만을 고려해 확률을 계산

수식 6.4 N-gram에서의 조건부 확률

$$P(w_t | w_{t-1}, w_{t-2}, \dots, w_{t-N+1})$$

TF-IDF

: 텍스트 문서에서 특정 단어의 중요도를 계산하는 방법

- 문서 내에서 단어의 중요도를 평가하는데 사용되는 통계적인 가중치를 의미
- BoW(Bag-of-Words)에 가중치를 부여하는 방법
 - 문서나 문장을 단어의 집합으로 표현하는 방법. 문서나 문장에 등장하는 단어의 중복을 허용해 빈도를 기록

표 6.2 BoW 벡터화

	I	like	this	movie	don't	famous	is
This movie is famous movie	0	0	1	2	0	1	1
I like this movie	1	1	1	1	0	0	0
I don't like this movie	1	1	1	1	1	0	0

- BoW를 이용해 벡터화 → 모든 단어는 동일한 가중치를 가짐

단어 빈도(TF)

: 문서 내에서 특정 단어의 빈도수

수식 6.5 단어 빈도

$$TF(t,d) = count(t,d)$$

문서 빈도(DF)

: 한 단어가 얼마나 많은 문서에 나타나는지 의미

수식 6.6 문서 빈도

$$DF(t,D) = count(t \in d: d \in D)$$

역문서 빈도(IDF)

: 전체 문서 수를 문서 빈도로 나눈 다음에 로그를 취한 값 ⇒ 문서 내에서 특정 단어가 얼마나 중요한지 나타냄

- 문서 빈도가 높을수록 해당 단어가 일반적으로 상대적으로 중요하지 않다는 의미

- 문서 빈도의 역수를 취하면 단어의 빈도수가 적을 수록 IDF 값이 커지게 보정하는 역할

수식 6.7 역문서 빈도

$$IDF(t, D) = \log \left(\frac{count(D)}{1 + DF(t, D)} \right)$$

- IDF는 분모의 DF 값에 1을 더하는데, 이는 분모가 0이 되는 결과를 방지함
- log를 취해 정교한 가중치를 얻음

TF-IDF

:앞선 문서 빈도와 역문서 빈도를 곱한 값으로 사용

수식 6.8 TF-IDF

$$TF-IDF(t, d, D) = TF(t, d) \times IDF(t, d)$$

- 문서 내에 단어가 자주 등장하지만, 전체 문서 내에 해당 단어가 적게 등장하면 TF-IDF 값이 커짐
 - 전체 문서에서 자주 등장할 확률이 높은 관사나 관용어 등의 가중치 낮아짐